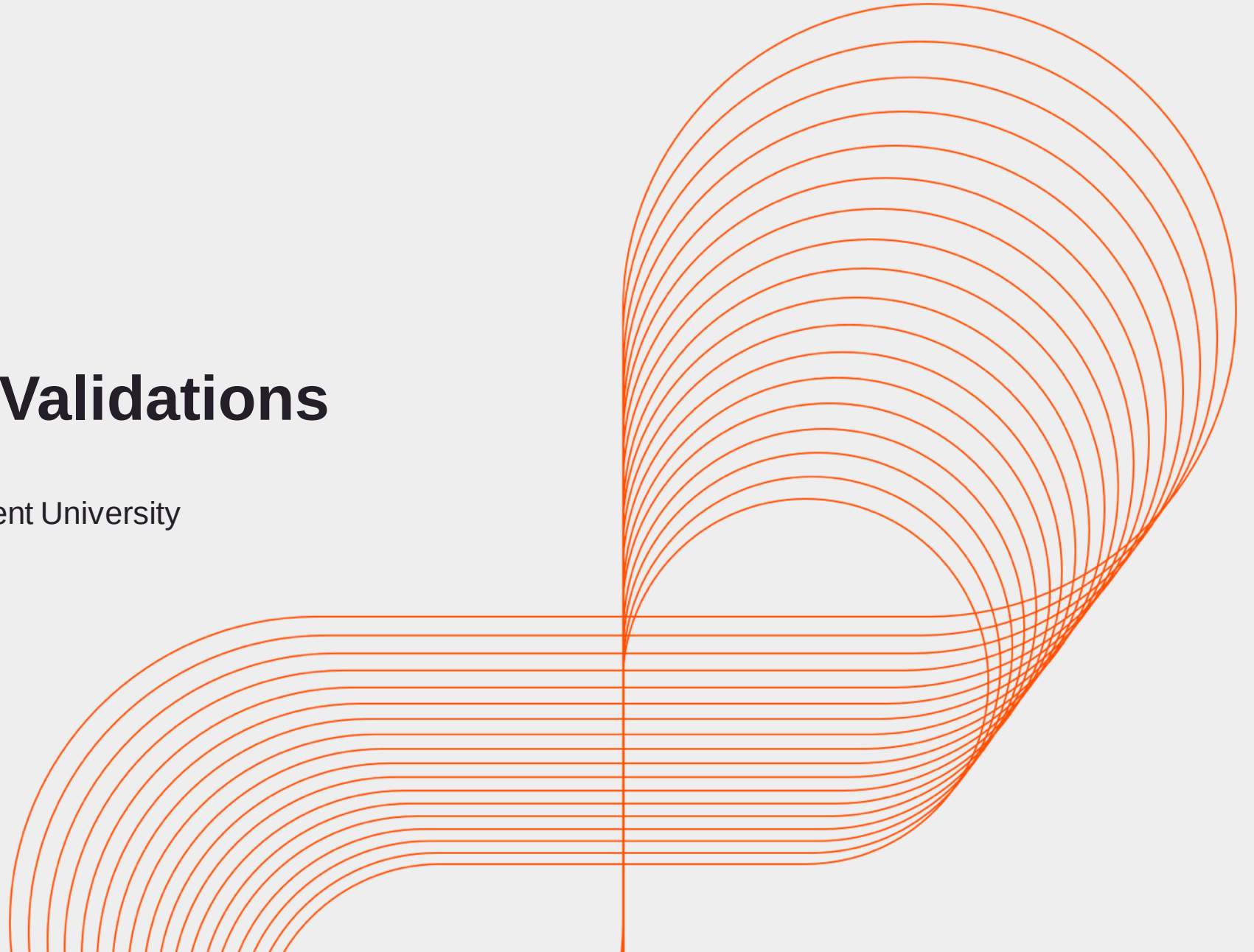




Custom Form Validations

Persistent Interactive | Persistent University



Key Learning Points

- Built-in Validators
- Custom Validations in Reactive forms
- Custom directives with Custom Validations
- Form Update Options

Built-in Validators

- Angular comes with a subset of built-in validators listed below.
 1. **required** - Requires a form control to have a non-empty value
 2. **minlength** - Requires a form control to have a value of a minimum length
 3. **maxlength** - Requires a form control to have a value of a maximum length
 4. **pattern** - Requires a form control's value to match a given regex
- For any other type of validation, one needs to go for writing a custom validator.

Custom Validation for Reactive Forms

- Lets create a custom validator for checking email for Reactive forms.

```
import { FormControl } from '@angular/forms';

function validateEmail(c: FormControl) {
  let emailExp = "expression"
  return emailExp.test(c.value) ? null : {
    validateEmail: {
      valid: false
    }
  };
}
```

Apply it to the Form Control Object

- Add the validation function name as second parameter in the form control.

```
ngOnInit() {  
  this.form = new FormGroup({  
    ...  
    email: new FormControl("", validateEmail)  
  });  
}
```

- If there are multiple validations to be added, then use below syntax.

```
email: new FormControl("",  
[Validators.required,validateEmail])
```

Custom Validator for Template Driven Forms

- For Template driven forms, we need to create a custom directive.
- The custom directive will contain the same logic as the function validateEmail.

```
import { Directive } from '@angular/core';
```

```
@Directive({  
  selector: '[validateEmail][ngModel]'  
})  
export class EmailValidator {}
```

Directive name

Add the directive to NG_VALIDATORS

- We basically add a new value to the NG_VALIDATORS token

```
import { NG_VALIDATORS } from '@angular/forms';
@Directive({
  selector: '[validateEmail][ngModel]',
  providers: [
    { provide: NG_VALIDATORS, useValue: validateEmail, multi: true }
  ]
})
class EmailValidator {}
```

NG_VALIDATORS

- Angular has an internal mechanism to execute validators on a form control.
- It maintains a multi provider for a dependency token called NG_VALIDATORS.
- It turns out that all built-in validators are already added to the NG_VALIDATORS token.
- So whenever Angular instantiates a form control and performs validation, it basically injects the dependency for the NG_VALIDATORS token, which is a list of all validators, and executes them one by one on that form control.

Declare the Directive

- Just like all components part of a module are listed in the declarations array, directives need to be declared too.

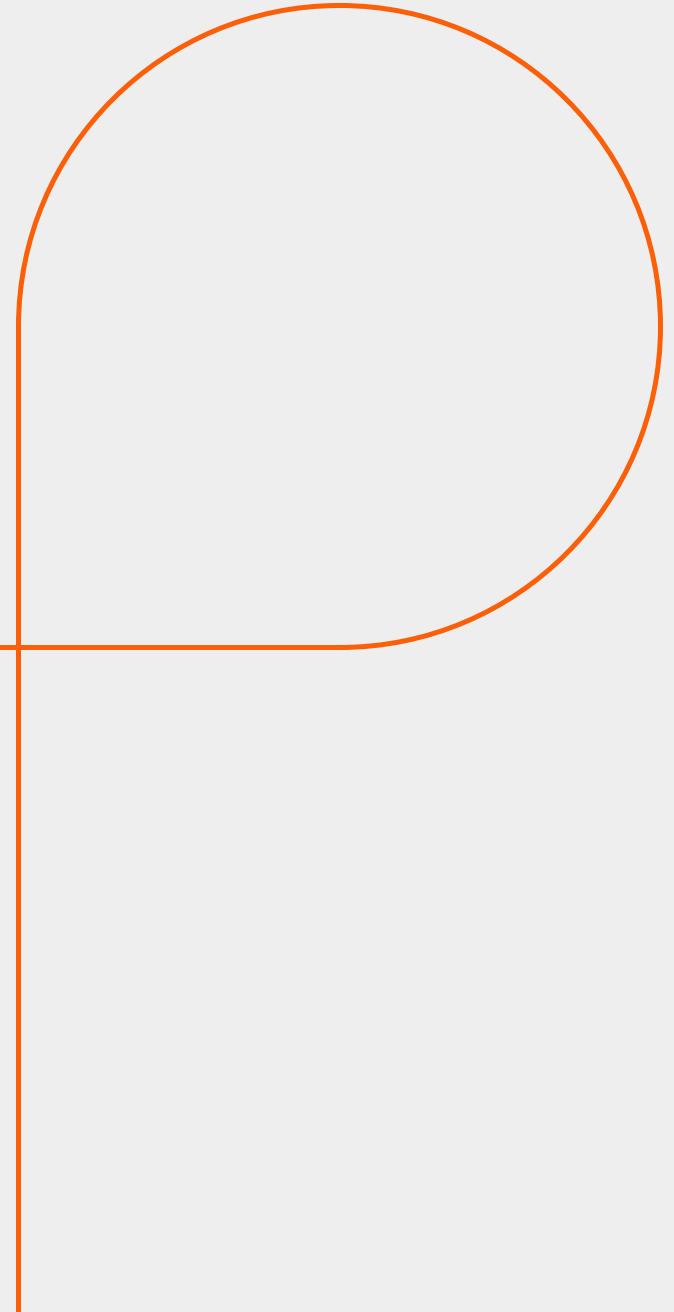
```
@NgModule({  
  ...  
  declarations: [AppComponent, EmailValidator],  
})  
export class AppModule {}
```

Apply the directive

- Apply the `validateEmail` directive to the template.

```
<form novalidate>  
  ...  
  <input type="email" name="email" ngModel validateEmail>  
</form>
```

Forms Update



Form Update Options

- Changes to a form/form control can be captured using any of the below options.
 1. **change**: change is the **default mode**. By using this update option the form / form control is updated after every single change.
 2. **blur**: the blur change mode is only updated the from values / validity status after a form control lost the focus.
 3. **submit**: updates are only done after form submit.

Template driven forms

- In template driven forms, the form update can be handled as below.

```
<input [(ngModel)]="lastname" [ngModelOptions]="{ updateOn: 'blur' }">
```

- To set the update mode on form level you can apply the **ngFormOptions** directive to the <form> element and set the updateOn property

```
<form [ngFormOptions]="{ updateOn: 'submit' }">
```

Reactive Forms

- Set the update mode on form control object.

```
lastname: new FormControl("", {  
  validators: Validators.required,  
  updateOn: 'submit'  
})
```

Form Group

- Set the update mode on form group.

```
this.nameForm = new FormGroup ({  
  firstname: new FormControl("", {  
    validators: Validators.required  
  }),  
  lastname: new FormControl("", {  
    validators: Validators.required  
  })  
}, { updateOn: 'blur' });
```

Summary: Session

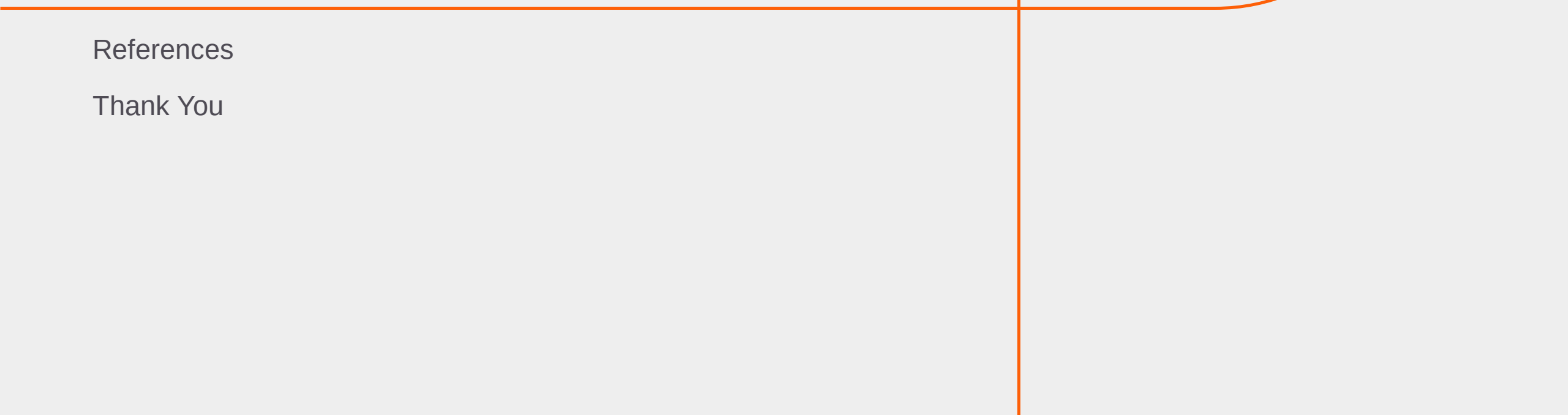
With this we have come to an end of our session, where we discussed about

- Built-in Validators
- Custom Validations in Reactive forms
- Custom directives with Custom Validations
- Form Update Options

References

- Custom Validations
 - <https://codeburst.io/angular-4-custom-validation-for-template-based-forms-b7dee9be48a6>
 - <https://blog.thoughttram.io/angular/2016/03/14/custom-validators-in-angular-2.html>
- Set Update mode to blur or submit
 - <https://codingthesmartway.com/angular5-forms-update/>
- Better Errors Display
 - <https://www.toptal.com/angular-js/angular-4-forms-validation>
 - <https://coryryan.com/blog/angular-form-builder-and-validation-management>

Appendix



References

Thank You

Key Contacts :

Persistent University :

- Archana Ghatigar
archana_ghatigar@persistent.co.in



Thank you!

Persistent Interactive | Persistent University

